

APIs, Numpy & Pandas

January 2026

APIs, Files, and Analysis

Welcome to Week 4, where we bring together three powerful concepts that form the foundation of modern data work. This week, you'll learn to navigate your computer's file system, communicate with external services through APIs, and analyze data like a professional data scientist.

By the end of this module, you'll have the complete toolkit to extract data from anywhere, transform it into insights, and save your results—the full data pipeline from start to finish.



 MODULE 1

The OS Module: Navigating Your Computer



File System Navigation

Learn to find and locate files on your computer using Python's built-in tools.



Working Directory

Understand where Python is "sitting" and how to move around your folders.



Cross-Platform Paths

Build file paths that work on Windows, Mac, and Linux automatically.

Before we can load files or analyze data, Python needs to know where to find them. The `os` module is your guide to the file system—think of it as Python's GPS for navigating folders and files on any operating system.

Where Are We? The os Module

Before we load files, Python needs to know where they are. We use the `os` (Operating System) module to talk to Windows, Mac, or Linux—it's the universal translator between Python and your computer's file system.

```
import os
```

Every Python session has a "Current Working Directory": the folder it's currently sitting in. Think of it like your home base. When you ask Python to open "data.csv", it looks in this folder first.

 **Key Concept:** Jupyter and Python always have a starting location. Understanding this location is the first step to avoiding the dreaded "FileNotFoundError".

"Where am I?" and "What is here?"

1

Get Current Working Directory

```
os.getcwd()
```

Tells you exactly which folder you're in right now. This is your starting point.

2

List Directory Contents

```
os.listdir()
```

Shows every file and folder in your current location. If your file isn't here, Python can't see it!

The Trap: If your file isn't in `listdir()`, Python can't open it! This simple check solves 90% of `FileNotFoundError` issues.

These two commands are your debugging best friends. Whenever you get a file error, run these first to understand where Python is looking and what it can actually see.

Building Paths Safely: `os.path.join`

Hardcoding file paths can lead to errors across different operating systems, as Windows uses backslashes (\) while Mac/Linux use forward slashes (/). The `os.path.join()` function ensures your scripts are **portable** by automatically using the correct path separator for the current system.

```
import os

# os.path.join handles separators automatically
path = os.path.join("users", "gemini", "documents", "data.csv")

print(path)
# On Windows: users\gemini\documents\data.csv
# On Mac/Linux: users/gemini/documents/data.csv
```

The `os` module isn't just for looking; it's for doing.

Task	Command
Create a folder	<code>os.mkdir("new_folder")</code>
Rename a file	<code>os.rename("old.txt", "new.txt")</code>
Check if file exists	<code>os.path.exists("data.csv")</code>
Delete a file	<code>os.remove("old_file.txt")</code>

APIs: Standard & Custom

The Waiter Analogy

APIs act like waiters—you place an order (request), and they bring you data from the kitchen (server).

Authentication Required

Most APIs require an API key—your digital passport that identifies you and tracks your usage.

Client Libraries

Wrapper libraries simplify complex APIs, letting you focus on results instead of technical details.

APIs as Abstraction

APIs are a prime example of **abstraction**, a core concept in Object-Oriented Programming (OOP). They hide the intricate implementation details—like how a server processes data, handles database queries, or manages authentication logic—and expose only a simple, user-friendly interface. Just as you use OOP methods without needing to know their internal workings, APIs allow your code to access powerful web services without understanding their underlying complexity. This bridges the gap between theoretical OOP principles and real-world software development.

APIs (Application Programming Interfaces) are how modern software communicates. They let your Python code talk to other services over the internet—from AI models to weather data to social media platforms.

Introduction to APIs: The Waiter

What is an API?

An API is a way for code to talk to other software over the internet. Think of it like a waiter in a restaurant:

- You (the customer) make a request
- The waiter takes it to the kitchen (server)
- The kitchen prepares your order (processes data)
- The waiter brings back your food (returns data)

Standard APIs

With standard APIs, we send a request and the server sends back raw data, usually in JSON format. We use Python's `requests` library to handle this communication:

```
import requests  
response = requests.get(url)  
data = response.json()
```


The API Key: Your Digital Passport

01

What is an API Key?

A long string of random characters that tells the server who you are. Think of it like a VIP wristband or credit card—it grants access and tracks your usage.

02

Why Keys Matter

Most APIs aren't open to the public. They require authentication to prevent abuse, track usage limits, and sometimes charge for access based on how much you use.

03

The Golden Rule

NEVER share your API key. If someone steals it, they can use your quota and potentially your money. Treat it like a password.

📌 **Pro Tip:** If you post code on GitHub, hide your keys first using environment variables or `.env` files. Never commit keys to version control!

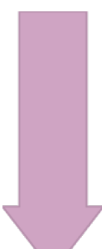
Using a "Client Library" (Gemini API)

Why Client Libraries?

Some APIs are complex (like AI services). Instead of dealing with raw HTTP requests, we use a **wrapper library** that simplifies everything.

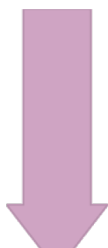
Example: Google's Gemini AI provides a Python library that handles all the technical details for you.

Three Simple Steps



Install the Library

```
pip install google-generativeai
```



Configure with Your Key

```
import google.generativeai as genai
genai.configure(api_key="YOUR_KEY_HERE")
```



Ask Questions!

```
model = genai.GenerativeModel('gemini-pro')
response = model.generate_content("Tell me a joke")
```

📌 Try running the code with a fake key first to show the "401 Unauthorized" error.

JSON: The Universal Language

What is JSON?

Most APIs send data back in **JSON** (JavaScript Object Notation) format. The good news? JSON looks exactly like a Python dictionary!

```
{  
  "name": "Alice",  
  "age": 30,  
  "city": "New York"  
}
```

It's just keys and values, organized in a structure Python already understands.

This seamless conversion makes working with APIs feel natural—you're just working with dictionaries and lists, data structures you already know.

Converting JSON to Python

When you receive data from an API, one simple command converts the web text into a Python dictionary:

```
data = response.json()
```

Now you can access the data like any dictionary:

```
print(data["name"]) # Alice  
print(data["age"]) # 30
```

Web Scraping: Your Web Data Extractor

While APIs offer structured access to data, not all websites provide them. Web scraping is the art of programmatically extracting information directly from web pages, turning unstructured HTML into usable data.

Think of it as having a super-fast assistant who can read a website and copy-paste exactly the pieces of information you need, but on a massive scale. Python, with its powerful libraries like `BeautifulSoup` and `Requests`, makes this process efficient and manageable.

Web Scraping in Action: Your First Extractor

Let's dive into a practical example. This Python script uses the `requests` library to fetch a webpage and `BeautifulSoup` to parse the HTML, allowing us to extract specific pieces of information, like the page title.

```
import requests
from bs4 import BeautifulSoup

# Define the URL of the page to scrape
url = 'http://example.com'

# Send an HTTP GET request to the URL
response = requests.get(url)

# Parse the HTML content of the page
soup = BeautifulSoup(response.text, 'html.parser')

# Find the <title> tag and extract its text
page_title = soup.find('title').get_text()

print(f"Extracted Page Title: {page_title}")
```

□ This simple script demonstrates the core steps of web scraping: fetching the HTML, parsing it, and then navigating the structure to pull out desired content. It's a powerful way to gather data not readily available via APIs.

MODULE 3

NumPy: The Engine

Speed

NumPy arrays are **fast**—they run at C-language speed, making calculations on large datasets nearly instantaneous.

Vectorization

Perform math on entire arrays at once without loops. `array * 2` multiplies every element instantly.

Dimensions

Work with multi-dimensional data: vectors (1D), matrices (2D), and tensors (3D+) for advanced applications.

NumPy is the computational engine that powers data science in Python. While lists are slow for mathematical operations, NumPy arrays are optimized for speed and efficiency. Think of NumPy as the engine—Pandas (which we'll cover next) is the car we actually drive.

Why NumPy? Speed Matters

The Problem with Lists

Python lists are flexible but slow for mathematical operations. To multiply every number in a list by 2, you need a loop:

```
numbers = [1, 2, 3, 4, 5]
result = []
for n in numbers:
    result.append(n * 2)
```

This works, but it's inefficient for large datasets.

The NumPy Solution

NumPy uses **vectorization**—math on the whole array at once, with no loops needed:

```
import numpy as np
numbers = np.array([1, 2, 3, 4, 5])
result = numbers * 2
```

This runs at C-language speed, making it hundreds of times faster for large datasets.



Installation: `pip install numpy` | **Import:** `import numpy as np`

Note: Arrays can only contain one data type. All values must be of the same data type.



Dimensions: Vectors vs. Matrices

1

1D Array (Vector)

A single row of numbers, like a list.

Shape: (3,)

```
np.array([1, 2, 3])
```

2

2D Array (Matrix)

Rows and columns, like a spreadsheet. Shape: (2, 3) means 2 rows, 3 columns

```
np.array([[1, 2, 3],
          [4, 5, 6]])
```

3

3D+ Array (Tensor)

A cube of numbers. Used in deep learning and AI for complex data like images or video.

```
np.array([[[1, 2], [3, 4]],
          [[5, 6], [7, 8]]])
```


Creating and Checking Dimensions

Creating a 2D Matrix

Use a list of lists to create a matrix. Each inner list becomes a row:

```
matrix = np.array([[1, 2],
                   [3, 4]])
```

This creates a 2×2 matrix with 2 rows and 2 columns.

The Most Important Command

`arr.shape` tells you the structure of your array. It returns a tuple showing dimensions:

```
print(matrix.shape) # (2, 2)
```

This means 2 rows and 2 columns.

 **Critical Skill:** Beginners often get "shape errors" in AI and data science. Learning `.shape` now saves massive headaches later. Always check your dimensions!

Indexing in 2D (Rows and Columns)

1

Single Element

Use `[row, column]` to get one value:

```
matrix[0, 1] # Row 0, Column 1
```

2

Entire Column

Use `:` to mean "all". Get all rows, specific column:

```
matrix[:, 0] # All rows, Column 0
```

3

Entire Row

Get specific row, all columns:

```
matrix[0, :] # Row 0, all columns
```

Analogy: Think of it like Battleship coordinates (A4, B6). `arr[row, col]` is just coordinate lookup in your data grid.

The comma syntax `[row, column]` is how you navigate 2D data. The colon `:` means "give me everything along this axis"—it's your wildcard for slicing entire rows or columns.

Creating and Inspecting Arrays

Create

```
arr = np.array([1, 2, 3])
```

Convert a Python list into a NumPy array for fast mathematical operations.

Check Shape

```
arr.shape
```

The most important command in AI. Returns dimensions as a tuple: (3,) for a 1D array.

Check Type

```
arr.dtype
```

Shows the data type: integers (`int64`), floats (`float64`), etc.

These three commands form your array inspection toolkit. Before performing any operations, check your shape to ensure your data has the structure you expect. Type checking helps catch errors where numbers are accidentally stored as text.

Common NumPy Methods (Math)



np.mean(arr)

Calculate the average of all values in the array. Essential for statistical analysis.



np.max(arr)

Find the highest number in the array. Useful for identifying peaks or outliers.



np.argmax(arr)

Return the *index location* of the highest number. Critical for classification tasks—finding the "winner".



np.sum(arr)

Add up all values in the array. The foundation for many statistical calculations.


AI Connection: `argmax` is crucial in machine learning. When a model predicts probabilities for different classes, `argmax` tells you which class won.

Generating Fake Data (Random)

Why Generate Data?

Sometimes you need data to test your code, but you don't have a real file yet. NumPy can create realistic test data instantly.



This is how you simulate games, create dummy datasets for practice, or generate test data for plotting. It's an essential tool for learning and prototyping before you have real data.

Random Methods

- **Random Integers**

```
np.random.randint(low, high, size)
```

Example: `dice_rolls = np.random.randint(1, 7, 100)`
 simulates 100 dice throws.

- **Random Decimals**

```
np.random.rand(rows, cols)
```

Generates random decimals between 0 and 1. Perfect for probabilities or test matrices.

 MODULE 4

Pandas: The Data Workstation

Tabular Data

Handles rows and columns like Excel, but with the power of Python code.

Universal Reader

Read almost any data format: CSV, Excel, JSON, SQL, HTML, and more.

Data Cleaning

Filter, transform, and clean messy real-world data with simple commands.

Analysis Tools

Built-in functions for grouping, aggregating, and statistical analysis.

"If you can do it in Excel, you can do it faster in Pandas."

Pandas is Python's answer to Excel—but supercharged. Install with `pip install pandas` and import with `import pandas as pd`.

Meet Pandas: Python's Excel

What is Pandas?

Pandas is the go-to library for working with tabular data in Python. It provides powerful tools for:

- Loading data from multiple formats
- Cleaning and transforming messy data
- Filtering and selecting specific information
- Performing statistical analysis
- Saving results in various formats

```
pip install pandas
import pandas as pd
```

📄 **Key Concept:** A Pandas DataFrame is like an Excel spreadsheet in Python—rows, columns, and cells of data that you can manipulate with code.

Loading Data (More than just CSV)



CSV Files

```
df = pd.read_csv("data.csv")
```

The most common format. Comma-separated values that work everywhere.



Clipboard Magic

```
df = pd.read_clipboard()
```

Converts whatever you've copied into a DataFrame. Perfect for quick data from websites!



Excel Files

```
df = pd.read_excel("data.xlsx")
```

Read .xlsx files directly. Requires `openpyxl` library. Use the `sheet_name` argument to specify the sheet you want (either by indexing or using the name of the sheet).



Other Formats

`read_json`, `read_sql`, `read_html` (scrapes tables from websites!)

📄 **Party Trick:** `read_html` can grab tables directly from Wikipedia or any website. Try it on a Wikipedia page with tables!

Advanced Loading: Secure File Paths

Working with real-world data means dealing with file paths. Learn how to build them robustly and load your data without errors.



1. Construct the Path

Use `os.path.join()` to safely combine path components, ensuring cross-OS compatibility.

```
import os
base_path = r"C:\...\documentary"
file_path = os.path.join(base_path, "data.csv")
```



2. Verify Existence

Use `os.path.exists(file_path)` to check if the file is present before attempting to load.

```
if os.path.exists(file_path):
    df = pd.read_csv(file_path)
else:
    print("File not found!")
```



3. Load into Pandas

Pass the generated `file_path` to Pandas' `read_csv()` function to load your data.

```
try:
    df = pd.read_csv(file_path)
    print("Success! File Read")
except FileNotFoundError:
    print(f"Error: I couldn't find a file at {file_path}")
```

📌 **Why this approach is better:** Directly typing paths can lead to "escape character" errors (e.g., `\Users`) and makes your code inflexible if file locations change. Using `os.path.join` and a base path avoids these issues.

These methods prevent common errors and make your code more adaptable.

Advanced Loading: Handling Messy Files

Real-world data is rarely clean. Pandas' `read_csv` has over 50 options to fix weird files instantly. Here are the most important ones:

No Headers?

```
pd.read_csv("data.csv", header=None,
            names=["Name", "Age", "City"])
```

Use `header=None` to prevent row 1 from becoming column names. Assign your own names with the `names` parameter.

Weird Separator?

```
pd.read_csv("data.txt", sep="|")
pd.read_csv("data.tsv", sep="\t")
```

If your file uses pipes (|) or tabs instead of commas, specify the separator. `\t` means tab character.

Wrong Data Type?

```
pd.read_csv("data.csv",
            dtype={"ZipCode": str})
```

Force specific columns to be read as text to preserve leading zeros like "09010".

Inspecting Your Data

df.head()



Sneak peek at the first 5 rows. Use `head(10)` to see more.

```
df.head()
```

df.info()



Check data types and find empty cells (nulls). Spots if numbers are stored as text.

```
df.info()
```

df.describe()



Instant math stats: mean, max, min, standard deviation for all numeric columns.

```
df.describe()
```

These three commands should be your first step with any new dataset. They give you a complete picture of what you're working with: the structure, the data types, and the statistical distribution.


Pro Tip: Use `info()` to spot if numbers are accidentally stored as "Object" type (text). This causes math operations to fail.

Handling Missing Data (The NaNs)

The Problem

Real data has holes. In Python, an empty value is called **NaN** (Not a Number). Math with NaN fails:

$$10 + \text{NaN} = \text{NaN}$$

We must fix them before analysis.

Analogy

Think of NaN values as potholes in a road. You can either:

- **Close the road** (delete the row)
- **Fill the hole** (replace with a value)

Option 1: Delete

```
df.dropna()
```

Removes any row with missing data. Simple but you lose information.

Option 2: Fill

```
df.fillna(0)
df.fillna("Unknown")
df.fillna(df.mean())
```

Replace emptiness with 0, text like "Unknown", or the column average. Preserves your data.

 **Decision Guide:** Drop rows if missing data is rare. Fill values if you need to keep all records or if the missing data is systematic.

Selecting and Filtering



Select a Column

```
df["Age"]
```

Returns a single column as a Series (like a 1D array).



Filter Rows

```
df[df["Age"] > 25]
```

Returns only rows where the condition is True. Logic: "Show me the DataFrame WHERE Age is greater than 25."



Multiple Conditions

```
df[(df["Age"] > 25) & (df["City"] == "NYC")]
```

Use `&` for AND, `|` for OR. Wrap each condition in parentheses.

Remember: Conditions go inside square brackets `[]`. The condition creates a True/False mask that filters your data.

Working with Dates and Times

The Problem

When you load a CSV, dates usually look like text (strings). Python doesn't know "Jan 1" comes before "Feb 1" if it's just text—it would sort alphabetically!

The Fix

```
df["Date"] = pd.to_datetime(df["Date"])
```

Converts text to proper datetime objects. Now you can do magic:

- `df["Date"].dt.month` - Extract month number
- `df["Date"].dt.year` - Extract year
- `df["Date"].dt.day_name()` - Get "Monday", "Tuesday", etc.
- `df["Date"].dt.quarter` - Get Q1, Q2, Q3, Q4

📌 **Business Analysis:** This is crucial for any time-based analysis like "Sales by Month" or "Year-over-Year Growth". Always convert dates first!

Saving Your Work

After cleaning and analyzing data, you need to save your results. Pandas makes this simple:

```
df.to_csv("clean_data.csv", index=False)
```

Important: Always use `index=False` to prevent Python from adding an annoying extra number column (0, 1, 2...) to your file.

Other Save Formats

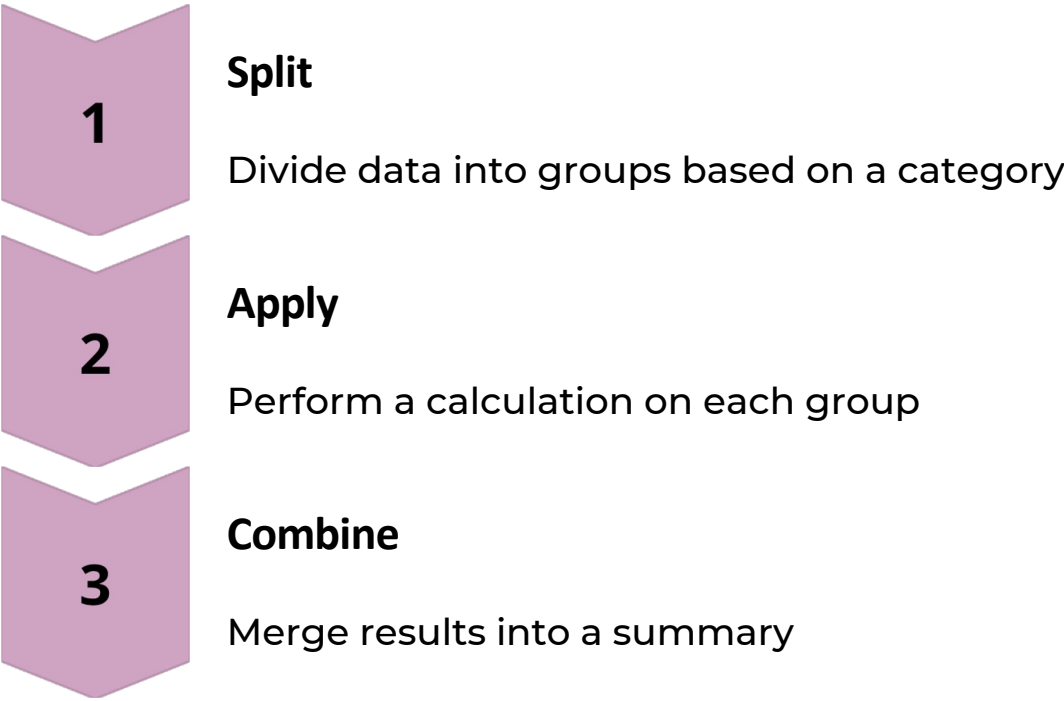
- `df.to_excel("data.xlsx")` - Save as Excel
- `df.to_json("data.json")` - Save as JSON
- `df.to_sql("table_name", connection)` - Save to database

You can verify the file was created using `os.listdir()` to close the loop!

GroupBy (The Pivot Table)

The `groupby` function is the most powerful analysis tool in Pandas. It answers questions like "What is the average salary per department?" or "Total sales by region?"

How It Works



Example

```
df.groupby("Department")["Salary"].mean()
```

This groups all employees by department, then calculates the average salary for each group.

Other Aggregations

- `.sum()` - Total
- `.count()` - Number of items
- `.max()` / `.min()` - Highest/Lowest
- `.std()` - Standard deviation

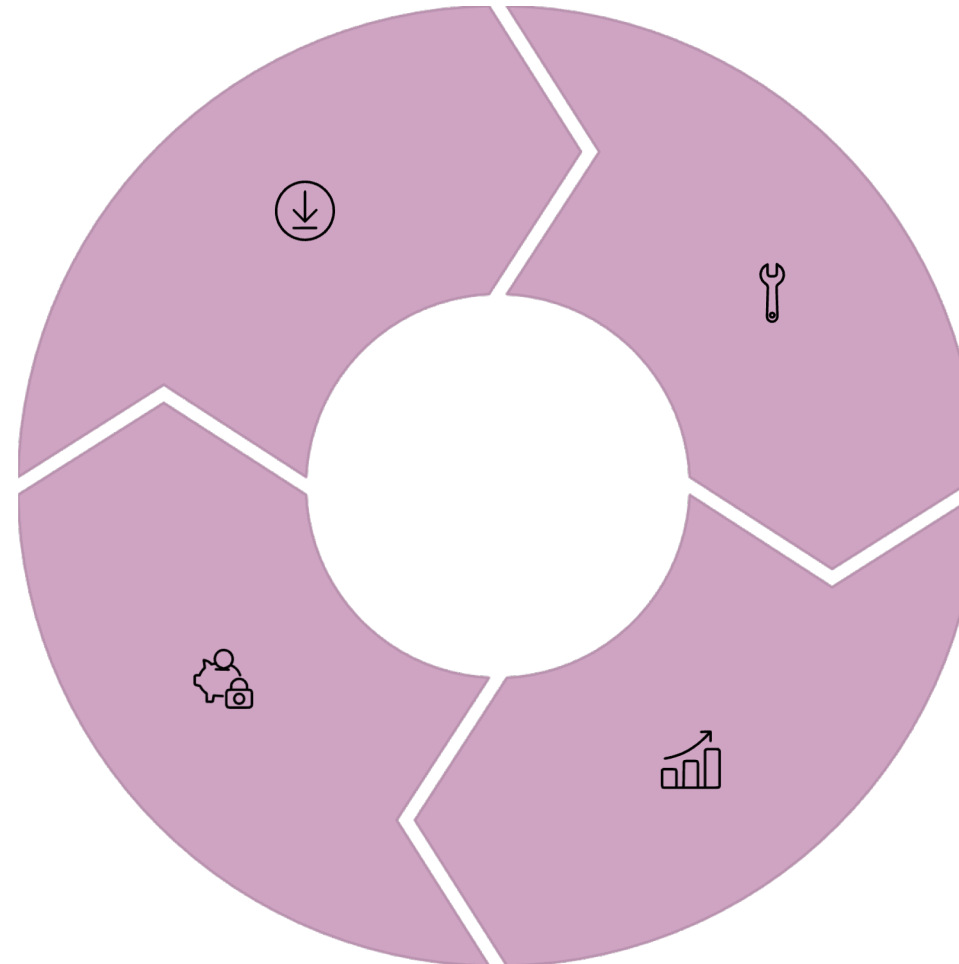
Analysis & Class Exercise

Extract
 Load data from various sources using
 Pandas readers

Transform
 Clean, filter, and reshape data to
 answer questions

Load
 Save results in the format needed for
 reporting or sharing

Analyze
 Use groupby, statistics, and
 aggregations to find insights



Now it's time to put everything together. You've learned the individual tools—now we'll combine them into a complete data analysis workflow.

Class Exercise: The Movie Database Analyst

Scenario: You've been hired to analyze a messy file containing thousands of movies. Your job is to find the best Action movies and create a curated list for a streaming service.

01

Load the Data

Import the file into a Pandas DataFrame.

02

Inspect and Clean

Check for missing values and remove incomplete records.

03

Filter for Quality

Find high-rated Action movies using conditional filtering.

04

Save Results

Export the curated list to a new CSV file.

📌 **Learning Goal:** This project proves you can complete the full ETL (Extract, Transform, Load) cycle—the foundation of data engineering and analysis.

What You've Accomplished

4

Modules Mastered

System navigation, APIs, NumPy, and Pandas

20+

New Commands

From `os.getcwd()` to `df.groupby()`

100%

Data Pipeline

Complete Extract-Transform-Load workflow



"You are now equipped with the fundamental tools of modern data science. These skills form the foundation for machine learning, AI, and advanced analytics."

Course Complete!

System Navigation

You can navigate file systems and build cross-platform paths with the `os` module.

API Communication

You can talk to AI services and external APIs, handling authentication and JSON data.

Data Analysis

You can load, clean, analyze, and save data using NumPy and Pandas.

These skills are the foundation for advanced topics like machine learning, artificial intelligence, data visualization, and statistical modeling. You've learned to work with real-world data, handle messy inputs, and produce clean outputs—the core competencies of a data professional.

The journey doesn't end here. Continue practicing with real datasets. Up next, we're exploring visualization libraries like Matplotlib and Seaborn, and diving into machine learning.

Resources:

API: <https://www.dataquest.io/blog/api-in-python/>

API: <https://www.dataquest.io/blog/api-skills-for-ai/>

Numpy: https://numpy.org/doc/2.4/user/absolute_beginners.html

Pandas: https://pandas.pydata.org/docs/getting_started/comparison/comparison_with_spreadsheets.html#compare-with-spreadsheets

Pandas Cheat Sheet: https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf

Pandas: <https://pandas.pydata.org/docs/reference/index.html>